

CHAPTER 36

Recurrent Neural Networks (RNNs)

Recurrent neural networks (RNNs) are another specialized scheme of neural network architectures. RNNs are developed to solve learning problems where information about the past (i.e., past instants/events) is directly linked to making future predictions. Such sequential examples play up frequently in many real-world tasks such as language modeling where the previous words in the sentence are used to determine what the next word will be. Also in stock market prediction, the last hour/day/week stock prices define the future stock movement. RNNs are particularly tuned for time series or sequential tasks.

In a sequential problem, there is a looping or feedback framework that connects the output of one sequence to the input of the next sequence. RNNs are ideal for processing 1-D sequential data, unlike the grid-like 2-D image data in convolutional neural networks.

This feedback framework enables the network to incorporate information from past sequences or from time-dependent datasets when making a prediction.

In this section, we will cover the broad conceptual overview of recurrent neural networks and in particular the Long Short-Term Memory RNN variant (LSTM) which is the state-of-the-art technique for various sequential problems such as image captioning, stock market prediction, machine translation, and text classification.

The Recurrent Neuron

The first building block of the RNN is the recurrent neuron (see Figure 36-1). The neurons of the recurrent network are entirely different from those of other neural network architectures. The key difference here is that the recurrent neuron maintains a memory or a state from past computations. It does this by taking as input the output of the previous instant y_{t-1} in addition to its current input at a particular instant x_t .

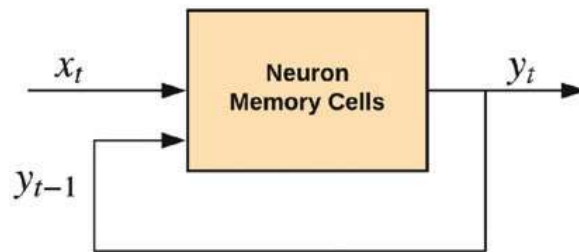


Figure 36-1. A recurrent neuron

In Figure 36-1, the recurrent neuron stands in contrast with neurons of the MLP and CNN architectures because instead of transferring a hierarchy of information across the network from one neuron to the other, data is looped back into the same neuron at every new time instant. A time instant can also mean a new sequence.

Hence, the recurrent neuron has two input weights, W_{x_t} and $W_{y_{t-1}}$, for the input at time x_t and for the input at time instant y_{t-1} . See Figure 36-2.

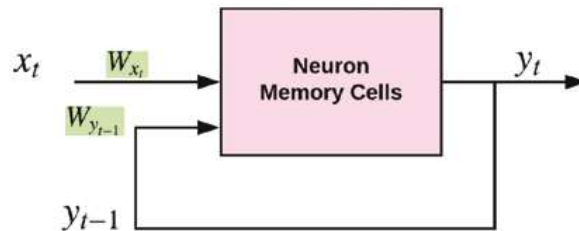


Figure 36-2. Recurrent neuron with input weights

Similar to other neurons, the recurrent neuron also injects non-linearity into the network by passing its weighted sums or affine transformations through a non-linear activation function.

Unfolding the Recurrent Computational Graph

A recurrent neural network is formalized as an unfolded computational graph. An unfolded computational graph shows the flow of information through the recurrent layer at every time instant in the sequence. Suppose we have a sequence of five time steps, we will unfold the recurrent neuron five times across the number of instants.

The number of sequences constitutes the layers of the recurrent neural network architecture. See Figure 36-3.

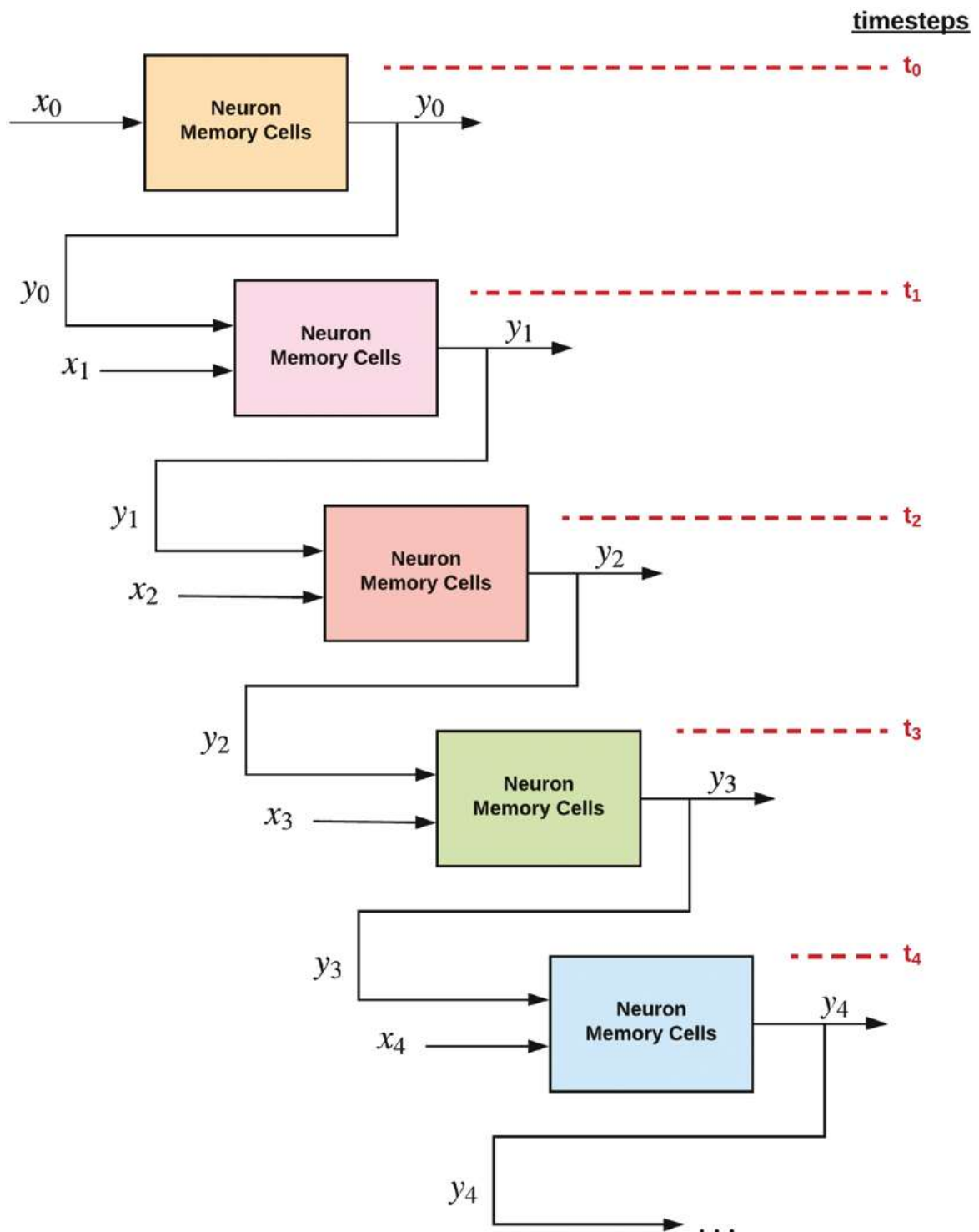


Figure 36-3. Unfolding the recurrent neuron into a recurrent neural network

From the unrolled graph of the recurrent neural network, we can observe how the input into the recurrent layer includes the output of the previous time step $t - 1$ in addition to the current input at time step t . This architecture of the recurrent neuron is central to how the recurrent neural network learns from past events or past sequences.

Up until now, we have seen that the recurrent neuron captures information from the past by storing memory or state in its memory cell. The recurrent neuron can have a much more complicated memory cell (such as the GRU or LSTM cell) than the basic RNN cell as illustrated in the images so far, where the output at time instant $t - 1$ holds the memory.

Basic Recurrent Neural Network

Earlier on, we mentioned that when a recurrent network is unfolded, we can see how information flows from one recurrent layer to the other. Further, we noted that the sequence length of the dataset determines the number of recurrent layers. Let’s briefly illustrate this point in Figure 36-4. Suppose we have a time series dataset of ten layers, for each row sequence in the dataset, we will have ten layers in the recurrent network system.

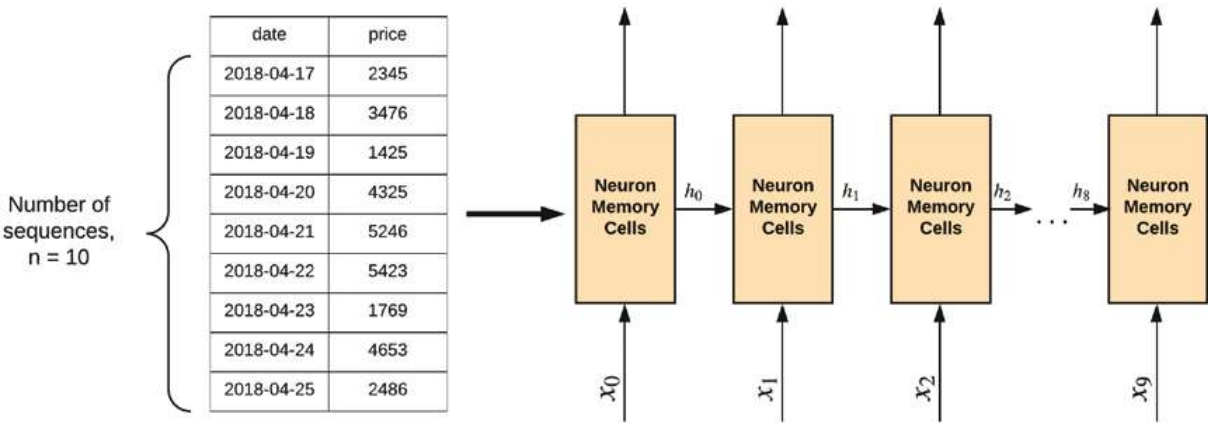


Figure 36-4. Dataset to layers

At this point, we must firmly draw attention to the fact that the recurrent layer does not comprise of just one neuron cell, but it is instead a set of neurons or neuron cells as shown in Figure 36-5. The choice of the number of neurons in a recurrent layer is a design decision when composing the network architecture.

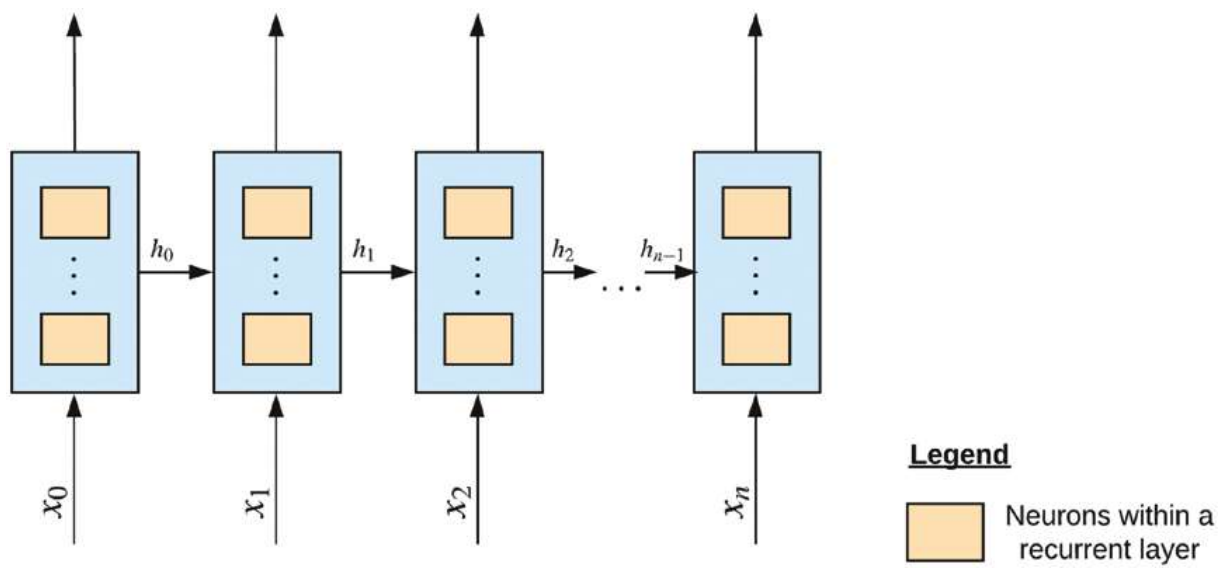


Figure 36-5. *Neurons in a recurrent layer*

Each neuron in a recurrent layer receives as input the output of the previous layer and its current input. Hence, the neurons each have two weight vectors. Again, just like other neurons, they perform an affine transformation of the inputs and pass it through a non-linear activation function (usually the hyperbolic tangent, $\tanh$). Still, within the recurrent layer, the output of the neurons is moved to a dense or fully connected layer with a softmax activation function for outputting the class probabilities. This operation is illustrated in Figure 36-6.

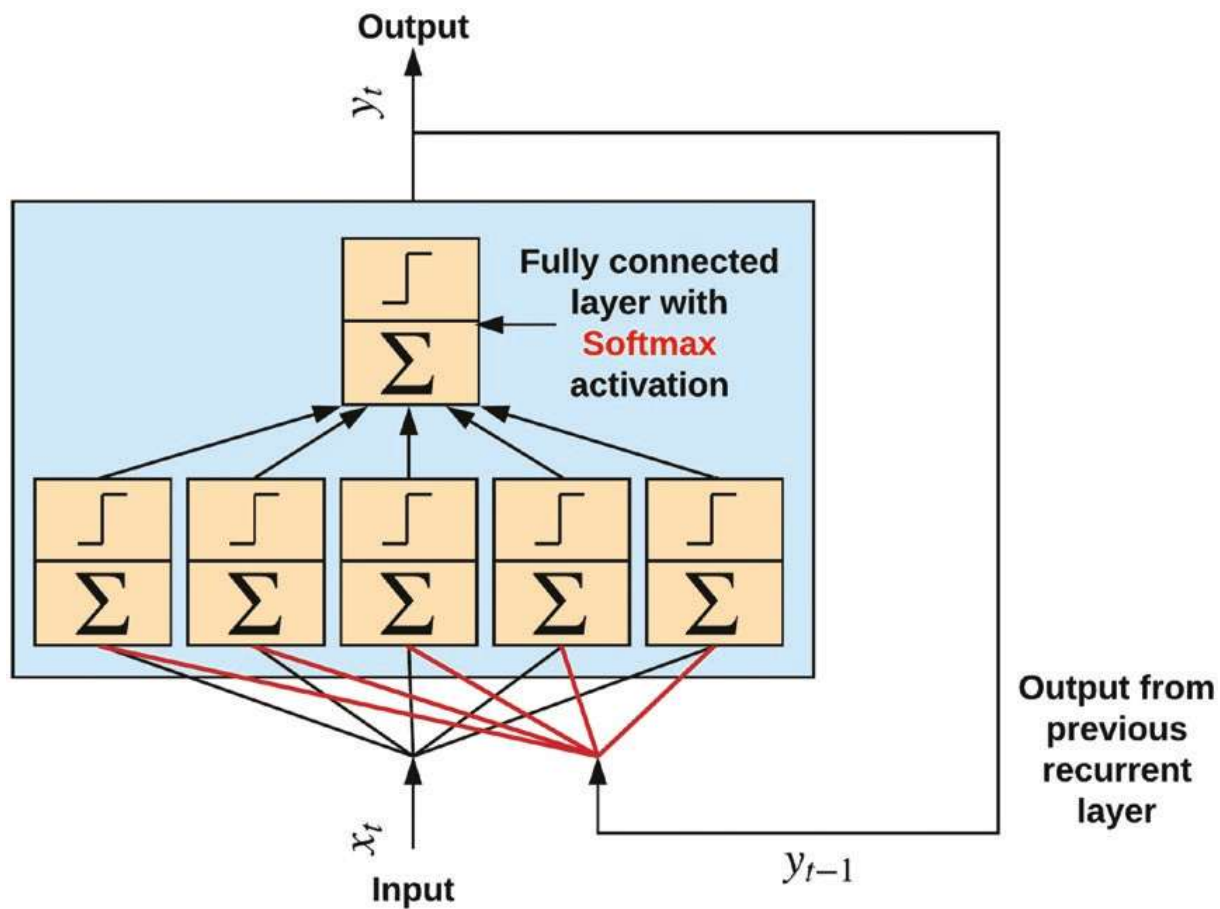


Figure 36-6. Computations within a recurrent layer

Recurrent Connection Schemes

There are two main schemes for forming recurrent connections from one recurrent layer to another. The first is to have recurrent connections between hidden units, and the other is recurrent connections between the hidden unit and the output of the previous layer. The different schemes are visually illustrated in Figure 36-7.

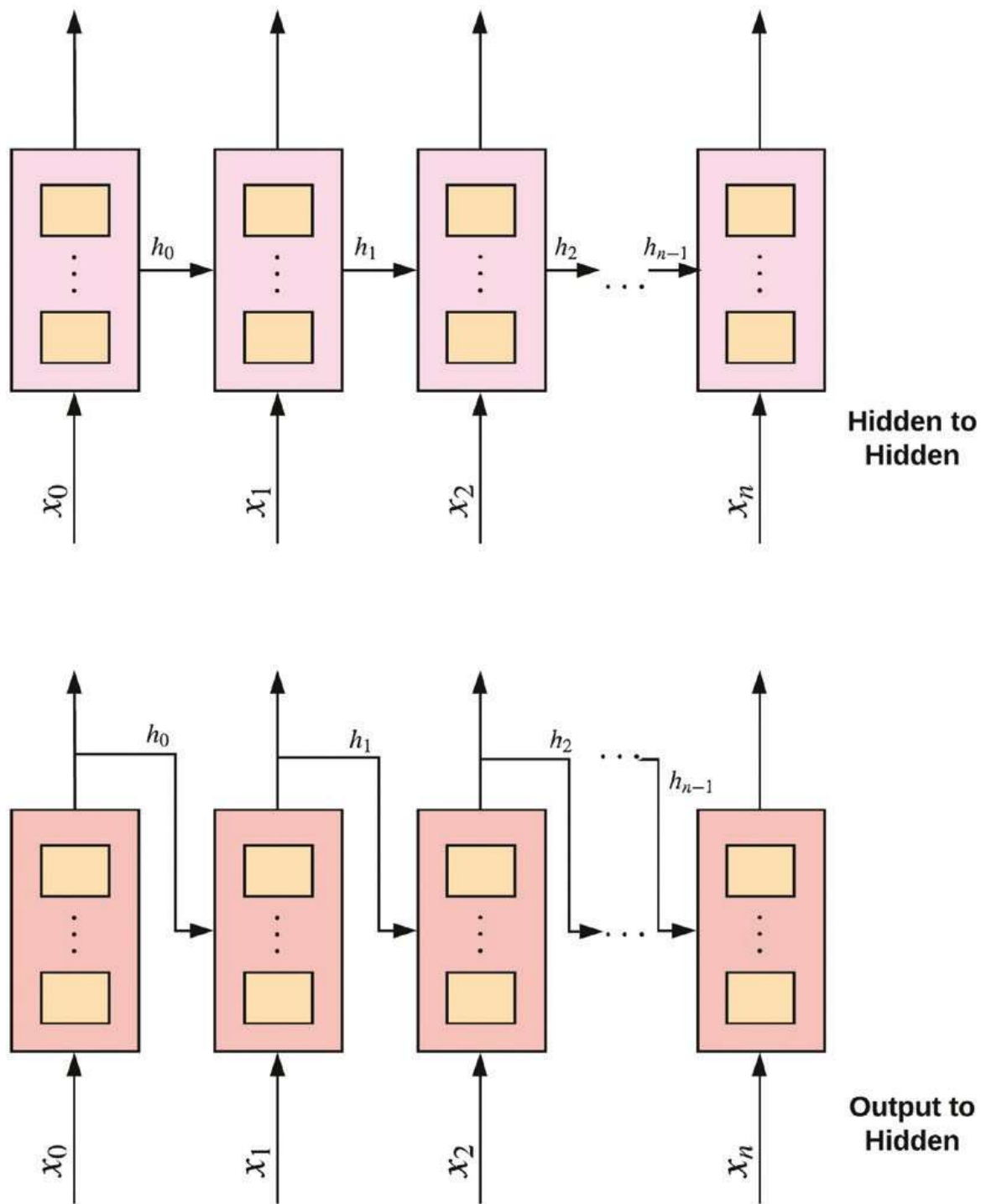


Figure 36-7. Recurrent connection schemes

The hidden-to-hidden recurrent configuration is found to be superior to the output-to-hidden form because it better captures the high-dimensional feature information about the past. In any case, the output-to-hidden recurrent form is less computationally expensive to train and can more easily be parallelized.

Sequence Mappings

Recurrent neural networks can represent sequence problems in a variety of ways. The flexibility of RNN mappings is that it operates on inputs and outputs of the network as sequences, thus freeing the network from the fixed sized input-output constraints found in other neural network architectures such as MLP and CNN.

Here are a few examples of varying sequence problems solved using RNNs:

1. An input to a sequence of output. This configuration is used for image captioning problems when an image is passed as an input to the network, and the output is a sequence of words. See Figure 36-8.

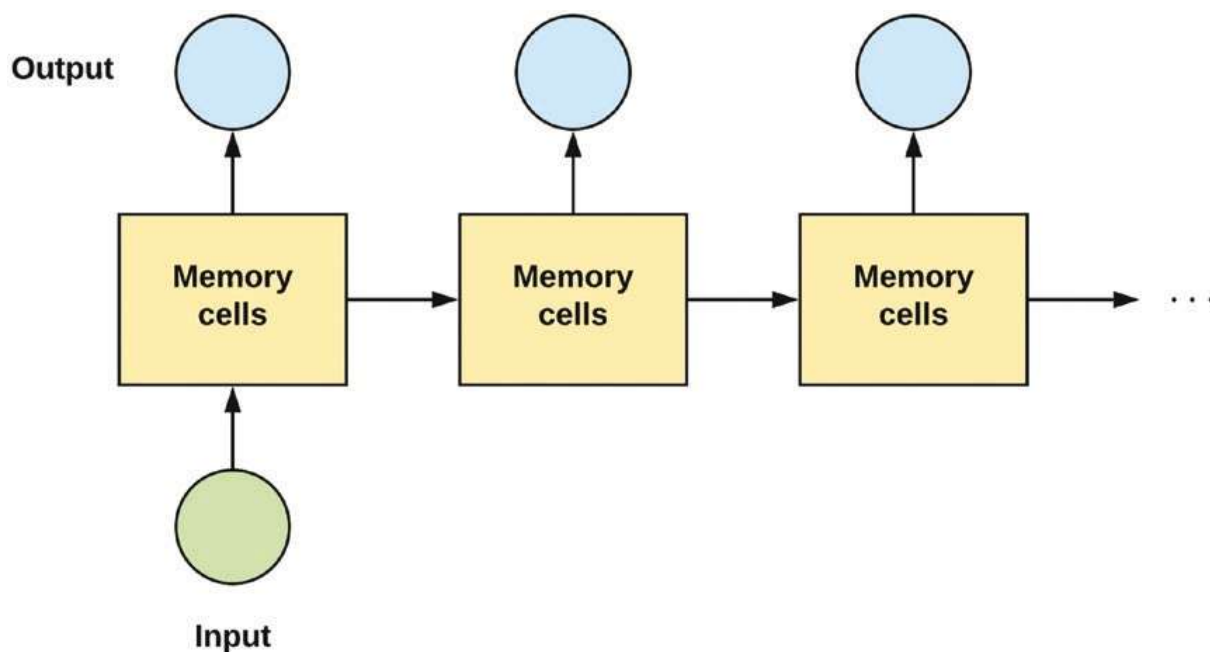


Figure 36-8. An input to a sequence of output

2. A sequence of inputs to an output. For example, in sentiment analysis, we need to pass in a sequence of words as input to the network, and the output is a class indicating either a positive or negative review or sentiment. See Figure 36-9.

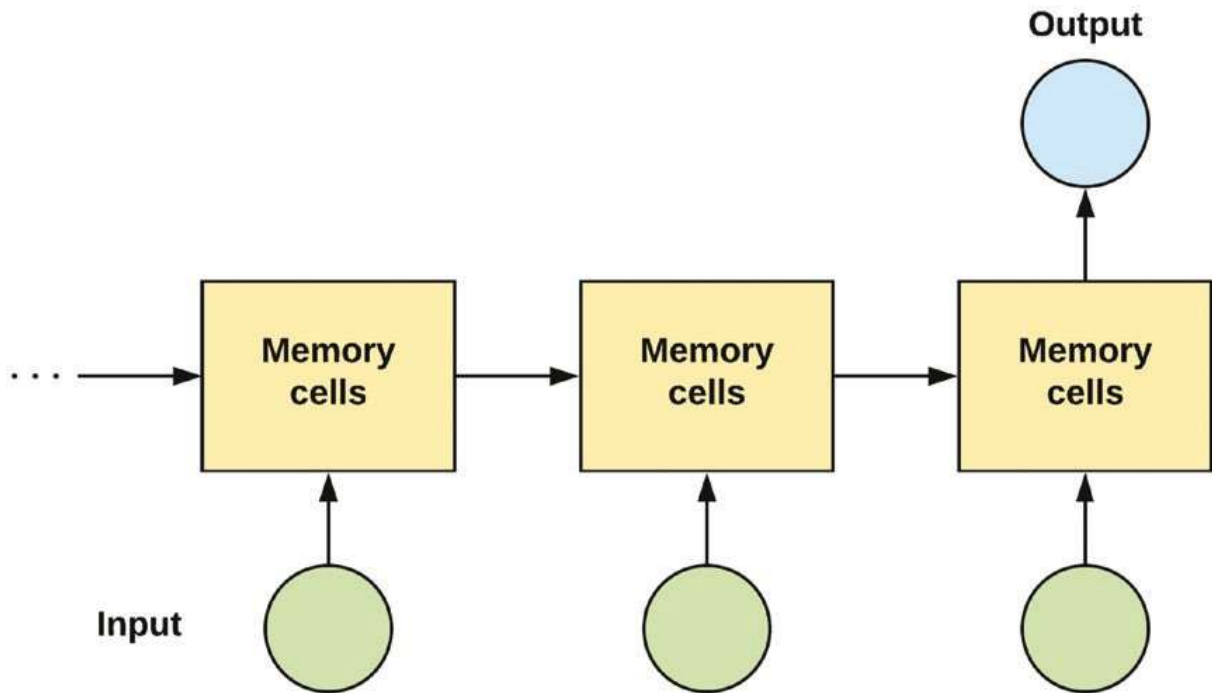


Figure 36-9. *A sequence of inputs to an output*

3. Sequence input to sequence output. This mapping operation is suited in application areas such as machine translation and speech recognition. It is more popularly called the encoder-decoder or sequence-to-sequence architecture. In this case, we may have a sequence of words in a particular language as input, and we want a sequence of words as output in another language. See Figure 36-10.

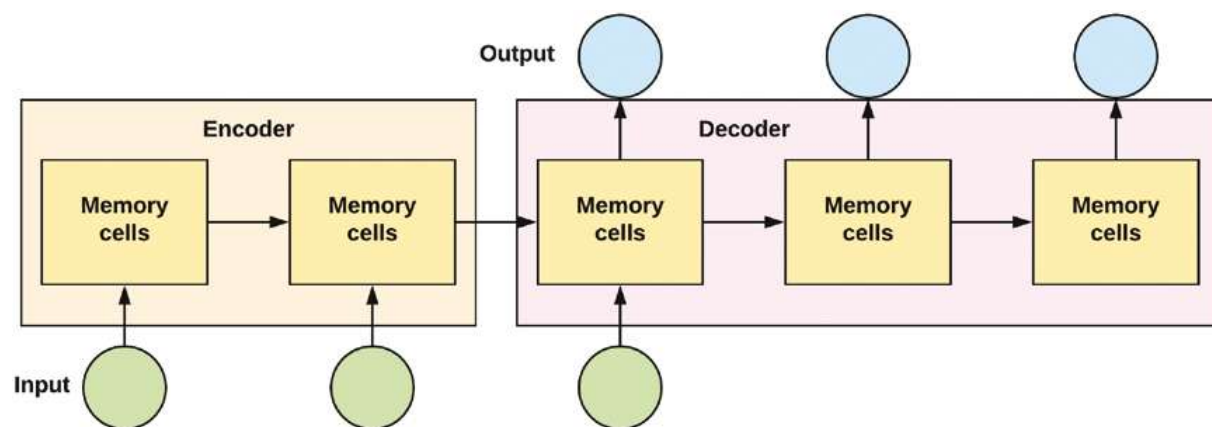


Figure 36-10. Sequence input to sequence output

4. Synced sequence input to output. This sort of framework is ideal for video classification in the event we want to label each video frame. See [Figure 36-11](#).

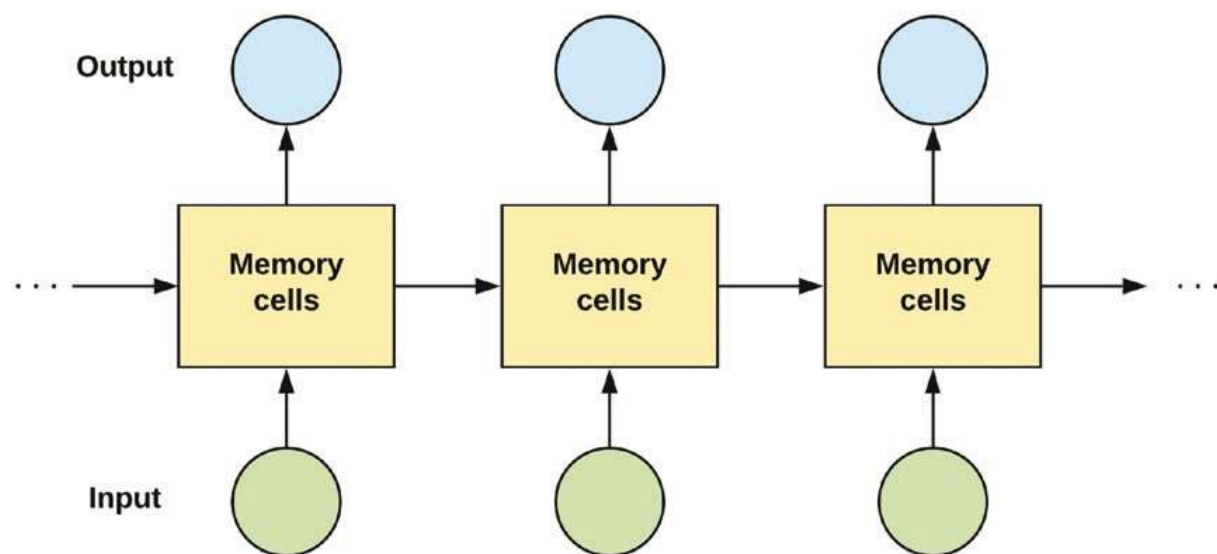


Figure 36-11. Synced sequence input to output

In the schemes illustrated in this sub-section, information flows from the hidden unit (or memory cell) of the recurrent layer at time instant $t - 1$ to the hidden unit at time instant t . As discussed earlier, this is because the transferred information is more feature-rich and contains more information from the past.

Training the Recurrent Network: Backpropagation Through Time

The recurrent neural network is trained in much the same way as other traditional neural networks by using the backpropagation algorithm. However, the backpropagation algorithm is modified into what is called backpropagation through time (BPTT).

Due to the architectural loop or recurrent structure of the recurrent network, vanilla backpropagation as is cannot work. Training a network using backpropagation involves calculating the error gradient, moving backward from the output layer through the hidden layers of the network and adjusting the network weights. However, this operation cannot work in the recurrent neuron because we have just one neural cell with recurrent connections to itself.

So, in order to train the recurrent network using backpropagation, we unroll the recurrent neuron across the time instants and apply backpropagation to the unrolled neurons at each time layer the same way it is done for a traditional feedforward neural network. This operation is further illustrated in Figure 36-12.

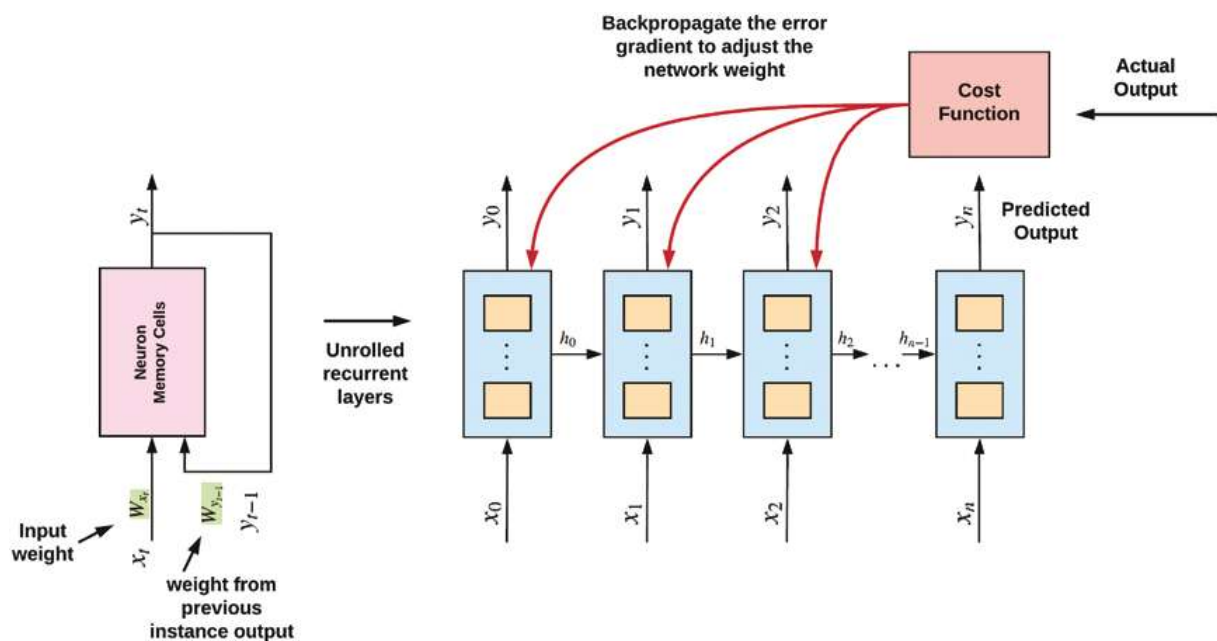


Figure 36-12. Backpropagation through time

A significant challenge of training the recurrent neural network is the vanishing and exploding gradient problem. When training a deep recurrent network for many layers of time instants, calculating the gradients of the weights of the neurons can become very volatile. When this happens, the value of the gradient can become extremely large tending to infinity, or they become tiny, all the way to zero. When this happens, the neurons become dead and cannot train or learn any new information further. This effect is called the exploding and vanishing gradient problem.

The exploding and vanishing gradient problem is most prevalent in recurrent neural networks because of the long-term dependencies or time instant of the unrolled recurrent neuron. A proposed alternative technique for mitigating this problem in recurrent networks (in addition to other discussed methods such as gradient clipping, batch normalization, and using a non-saturating activation function such as ReLu) is to discard early time instances or time instances in the distant past. This technique is called Truncated Backpropagation Through Time (truncated BPTT).

However, truncated BPTT suffers a significant drawback, and this is that some problems rely heavily on long-term dependencies to be able to make a prediction. A typical example is in language modeling where the long-term sequence of words in the past is vital in predicting the next word in the sequence.

The shortcoming of truncated BPTT and the need to deal with the problem of exploding and vanishing gradients led to the development of a memory cell called the Long Short-Term Memory or LSTM for short, which can store the long-term information of the problem in the memory cell of the recurrent network.

The Long Short-Term Memory (LSTM) Network

Long Short-Term Memory (LSTM) belongs to a class of RNN called gated recurrent unit. They are called *gated* because unlike the basic recurrent units, they contain extra components called gates that control the flow of information within the recurrent cell. This includes choosing what information to store in the cell and what information to discard or forget.

LSTM is very efficient for capturing the long-term dependencies across a large number of time instants. It does this by having a slightly more sophisticated cell than the basic recurrent units. The components of the LSTM are the

- Memory cell
- Input gate

- Forget gate
- Output gate

These extra components enable the RNN to remember and store important events from the distant past. The LSTM takes as input the previous cell state, c_{t-1} ; the previous hidden state, h_{t-1} ; and the current input, x_t . To keep in line with the simplicity of this book, we provide a high-level illustration of the LSTM cell showing how the extra components of the cell come together. In TensorFlow 2.0, LSTM layer is implemented in the method '**tf.keras.layers.LSTM()**'.

The illustration in Figure 36-13 is the LSTM memory cell. The components of the LSTM cell serve distinct functions in preserving long-term dependencies in sequence data. Let's go through them:

- The input gate: This gate is responsible for controlling what information gets stored in the long-term state or the memory cell, c . Working in tandem with the input gate is another gate that regulates the information flowing into the input gate. This gate analyzes the current input to the LSTM cell, x_t , and the previous short-term state, h_{t-1} .
- The forget gate: The role of this gate is to regulate how much of the information in the long-term state is persisted across time instants.
- The output gate: This gate controls how much information to output from the cell at a particular time instant. This gate controls the value of h_t (the short-term state) and y_t (the output at time t).

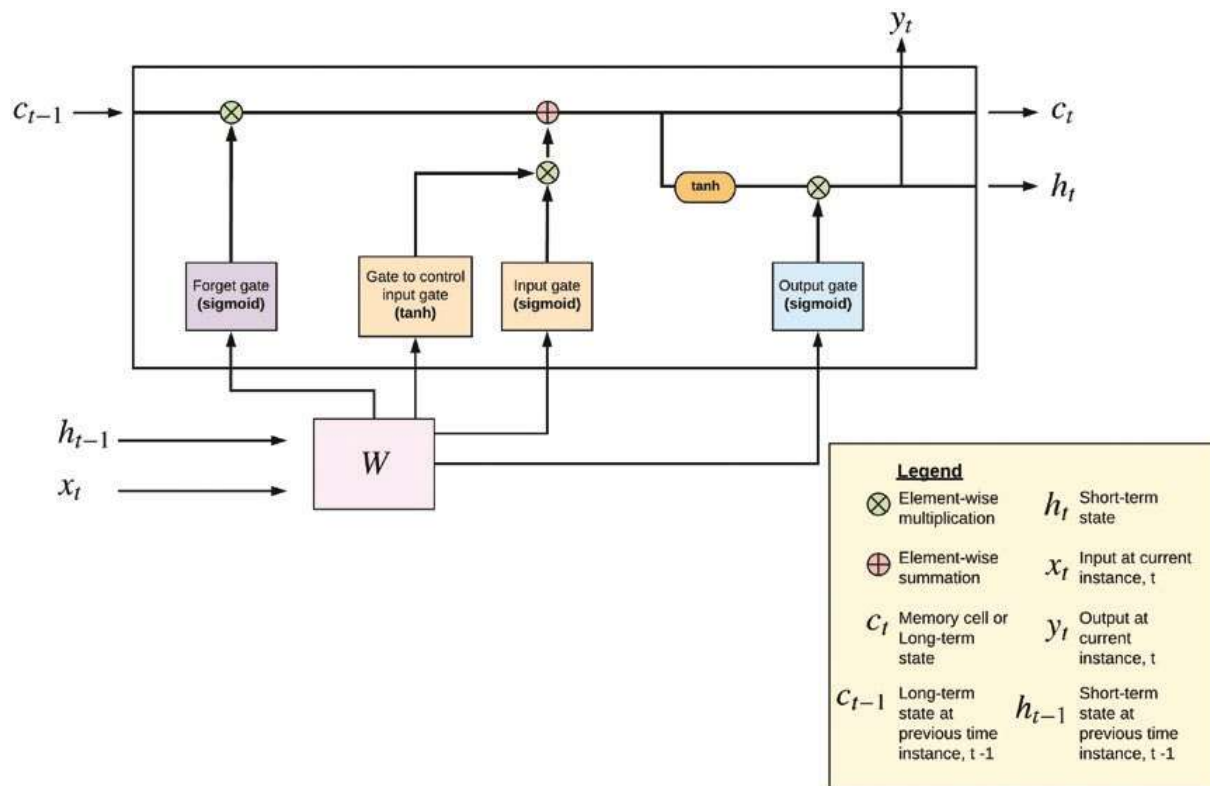


Figure 36-13. *LSTM cell*

It is important to note that the components of the LSTM cells are all fully connected neural networks. There exist other variants of recurrent networks with memory cells, two of such are the peephole connections and the gated recurrent units.

Peephole Connection

The peephole connection extends the LSTM network by also using information from the memory cell or long-term state of the previous time instant c_{t-1} as input to the LSTM gates. The goal of the peephole is to provide extra information into the LSTM unit by peeping at the stored long-term memory. This is further illustrated in Figure 36-14. In TensorFlow 2.0, the implementation of peephole connections to an LSTM layer is provided by the method '`tf.keras.experimental.PeepholeLSTMCell()`'.

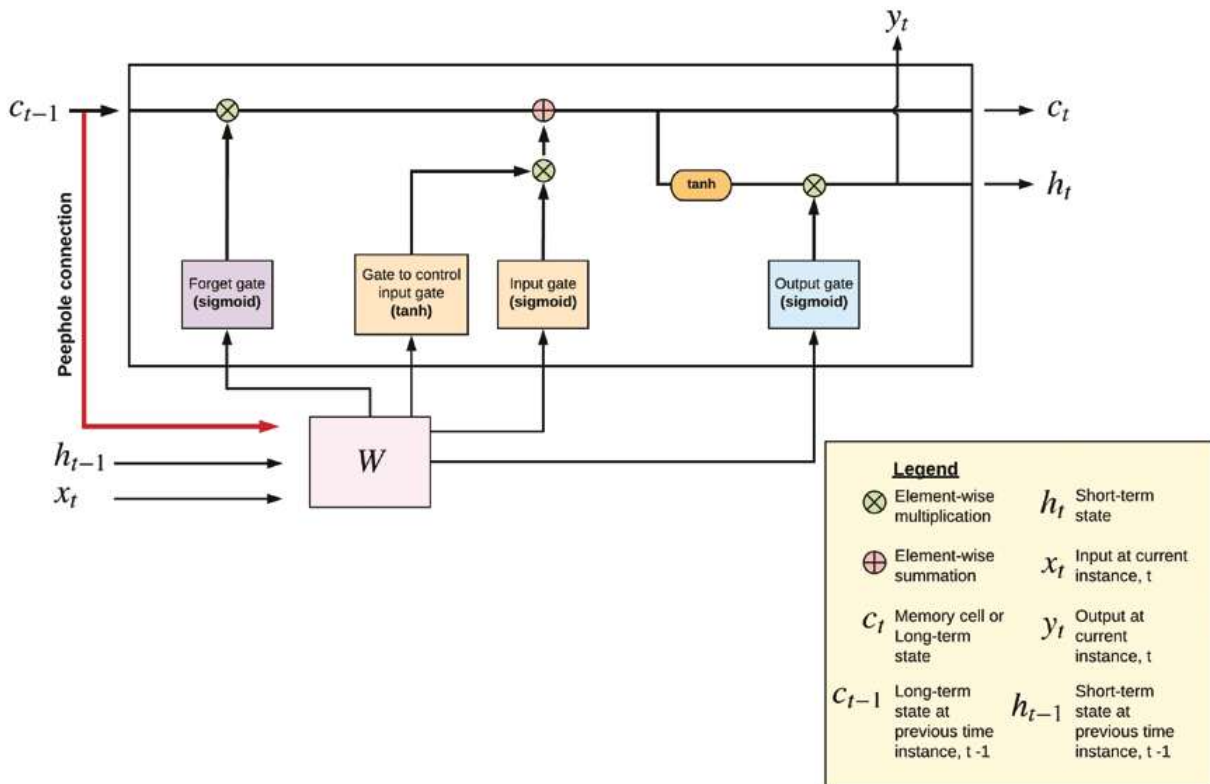


Figure 36-14. Peephole connection

Gated Recurrent Unit (GRU)

The gated recurrent unit (GRU) is a more recent recurrent neural network architecture than the LSTM, and it is also comparable simpler to implement with respect to the number of components within the unit and their operations. Despite its comparative simplicity, GRUs are high-performing recurrent architectures and, in most cases, even perform better than the LSTM in sequence modeling problems.

GRUs combine the forget and the input gates to decide on what information should be committed to the long-term memory or the memory cell and what information should be left out. Moreover, the GRU combines the cell (i.e., long-term state) and short-term states into a single state vector h_t . Also, the GRU removes the output gate and returns the state vector h_t at each time instant. This is further illustrated in Figure 36-15. In TensorFlow 2.0, the GRU layer is implemented in the method `'tf.keras.layers.GRU()'`.

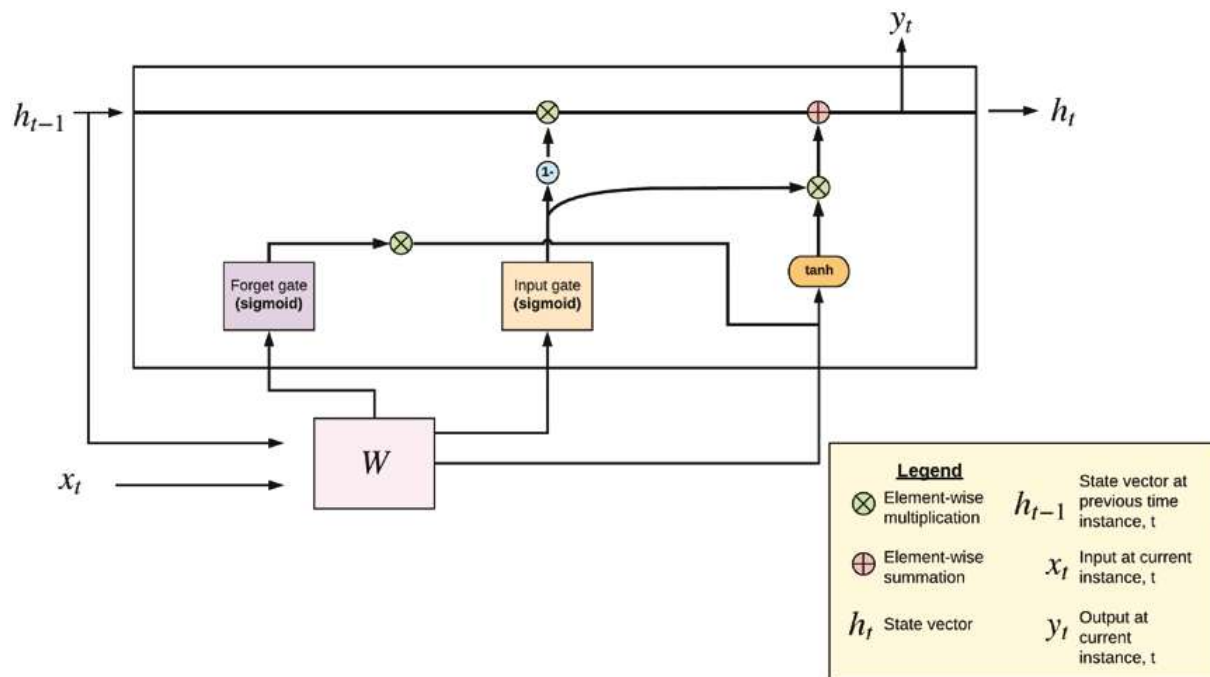


Figure 36-15. Gated recurrent unit

Recurrent Neural Networks Applied to Sequence Problems

Recurrent neural networks have many application areas for using LSTM models for sequence tasks. A couple of problems under this domain include sentiment analysis, machine translation, image captioning, video captioning, and voice recognition. As mentioned earlier, these problems can be modeled as a one-to-many model, a many-to-one model, or a many-to-many model. The section will survey a few LSTM architectures for tackling/modeling sequence problems:

- Long-term recurrent convolutional neural network, also known as CNN LSTM
- Encoder-Decoder LSTMs
- Bidirectional recurrent neural networks

Long-Term Recurrent Convolutional Network (LRCN)

The long-term recurrent convolutional network (LRCN) is a unique neural network architecture for generating descriptions of images and videos (which is seen as a sequence of images). These problems can be termed as visual time series modeling. The LRCN architecture combines the ability of the convolutional neural network (CNN) to extract image features together with a recurrent network for learning sequences or long-term dependencies. The LRCN passes visual inputs into a CNN to retrieve image features as outputs. These outputs are then passed into a recurrent LSTM network layer to generate the natural language descriptions. The recurrent layer can contain stacked LSTMs.

One core advantage of LRCN for modeling sequential vision problems such as image captioning and video captioning is that the network is not constrained to fixed lengths of inputs and outputs. Hence, it can be used to model sequential data with different lengths such as textual data and videos.

The following illustrations show how LRCN is applied to a variety of sequence problems:

1. **Image captioning:** Image captioning can be seen as a one-to-many sequence problem. The input is an image and therefore a static input, and the output is a sequence of text that describes the objects in the image; this is a sequential output. The use of LRCN for image captioning is illustrated in Figure 36-16.

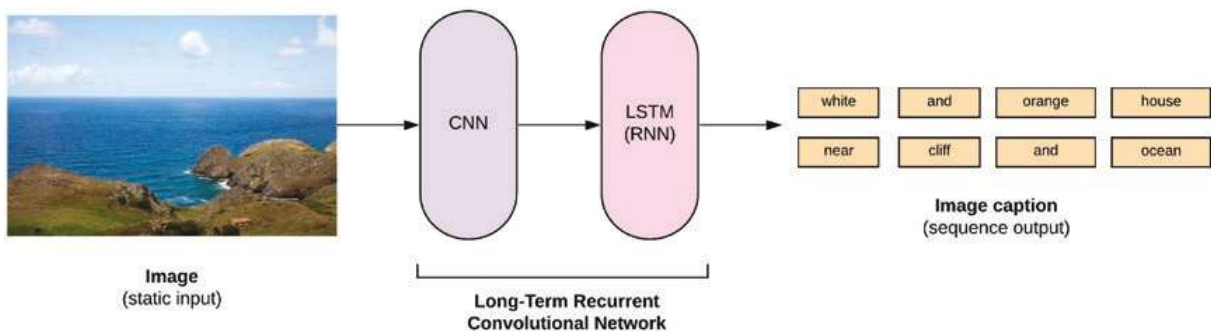


Figure 36-16. Image captioning (photo by Daniel Llorente on Unsplash)

2. Video captioning: Video can be seen as a sequence of images. Hence, in a video captioning problem, a sequence of images is passed as input to the LRCN model which in turn returns a sequence of outputs as a textual description for each video frame. Hence, video captioning can be seen as a many-to-many sequence problem. This approach is an example of an Encoder-Decoder LSTM where CNN is used as an image encoder that is initially trained for image classification. The final hidden layer, which is also called a bottleneck, is then passed as input to the RNN decode. It is typical to use an already pre-trained CNN on a large-scale image recognition task. A number of such models exist in the public domain. We will survey Encoder-Decoder LSTMs in more detail shortly. Video captioning is illustrated in Figure 36-17.

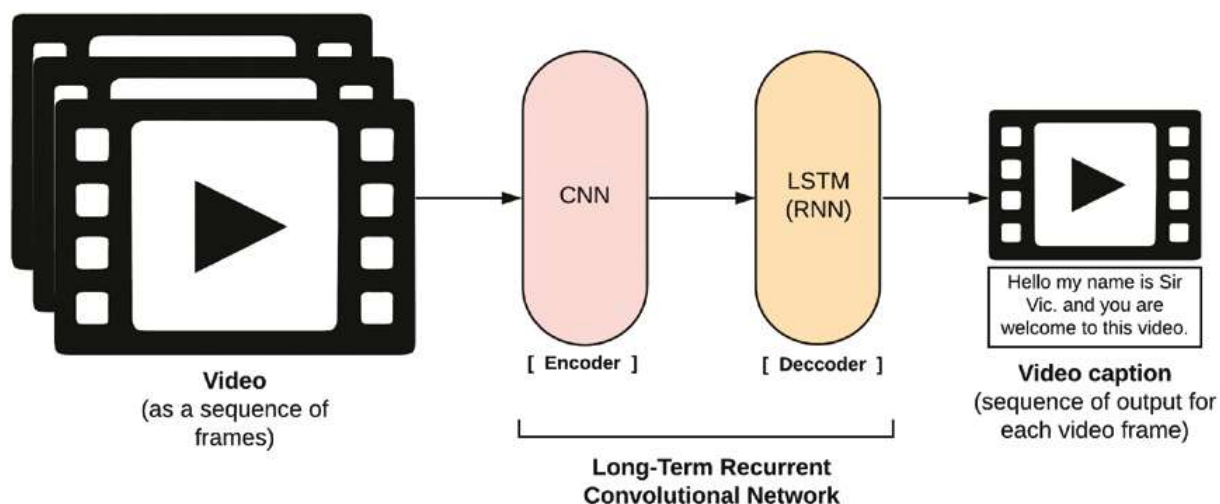


Figure 36-17. Video captioning

Encoder-Decoder LSTMs

Encoder-Decoder LSTM architecture handles a particular class of sequence problems that takes as input multiple time steps and also returns a multiple time step output. A major challenge of this sort of problems is that both the input and output sequences can have varied lengths.

The first part of the architecture, that is, the Encoder, is responsible for receiving and encoding the input sequence; the second part of the architecture, that is, the Decoder, takes in the output from the Encoder and then predicts the output sequence.

The sort of architecture is made for natural language processing problems where the output is a sequence of words. It is commonly used in machine translation, video captioning, and speech recognition. An illustration is already provided in Figure 36-10.

Bidirectional Recurrent Neural Networks

Bidirectional RNN is another particular type of recurrent neural network architecture that involves placing the recurrent layers beside each other where one layer works to learn the long-term dependencies from the past; this layer is called the forward LSTM. For the other layer, the input is reversed and fed into the network, so the network learns long-term dependencies from the future. This layer is called the backward LSTM. The bidirectional RNN is illustrated in Figure 36-18.

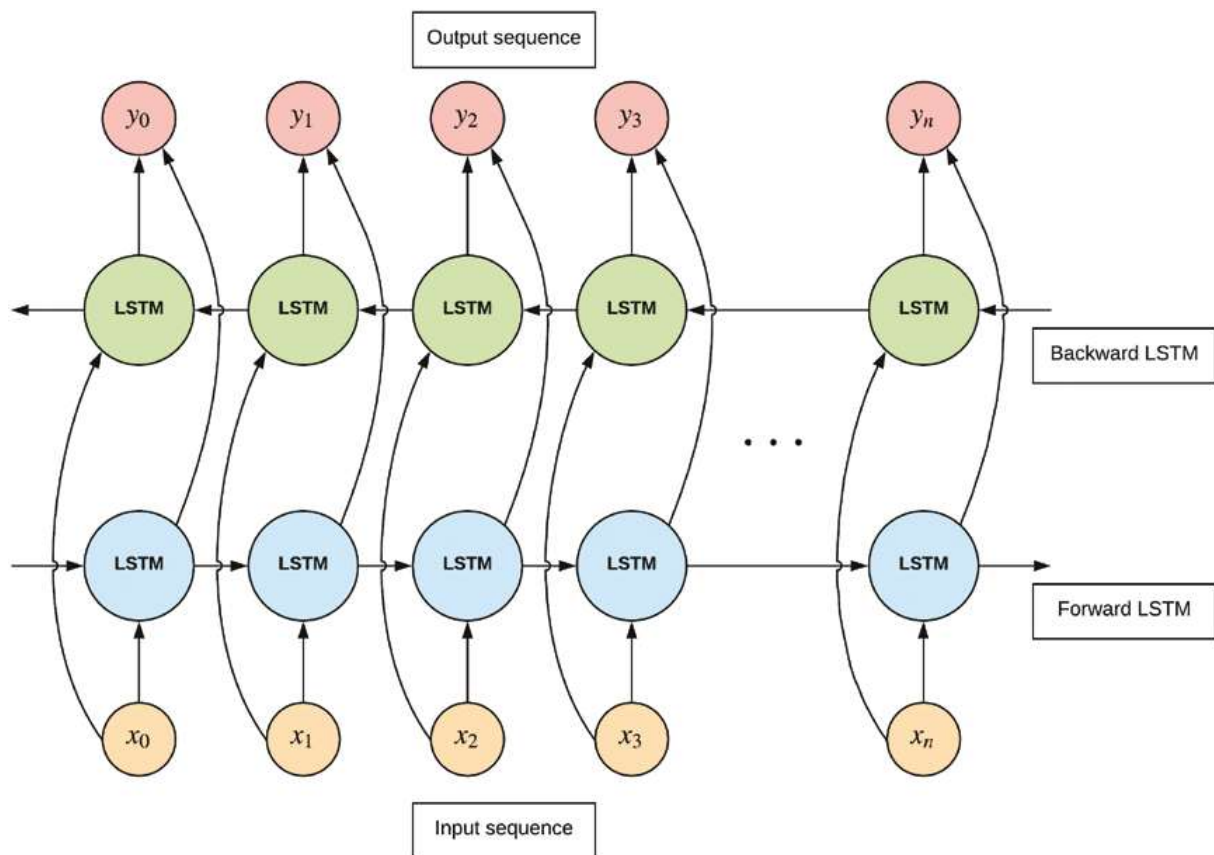


Figure 36-18. Bidirectional LSTM

When the outputs from these side-by-side networks are combined, it is easier to predict the next time step of a sequence having privy to the entire information gamut, because they process both information from the past and the future. Although this architecture was first designed for speech recognition tasks, it has performed impressively across a variety of other sequence prediction tasks. It is built to improve on the vanilla unidirectional LSTM which only has knowledge of the past.

This network is built on the understanding that some learning problems only make sense when a coherent set of information is present. For example, if a human interpreter is interpreting from one language to another, he first listens to a cohesive set of information in one language before interpreting to another language. This is because the context of an entire cohesive sentence gives the right basis for a correct interpretation.

RNN with TensorFlow 2.0: Univariate Timeseries

This section makes use of the Nigeria power consumption dataset to implement a univariate timeseries model with LSTM recurrent neural networks. The dataset for this example is the Nigeria power consumption data from January 1 to March 11 by Hipel and McLeod (1994), retrieved from DataMarket.

The dataset is preprocessed for timeseries modeling with RNNs by converting the data input and outputs into sequences using the method **'convert_to_sequences'**. This method splits the dataset into rolling sequences consisting of 20 rows (or time steps) using a window of **1**. In Figure 36-19, the example univariate dataset is converted into sequences of five time steps, where output sequence is one step ahead of the input sequence. Each sequence contains five rows (determined by the **time_steps** variable) and in this univariate case, 1 column.

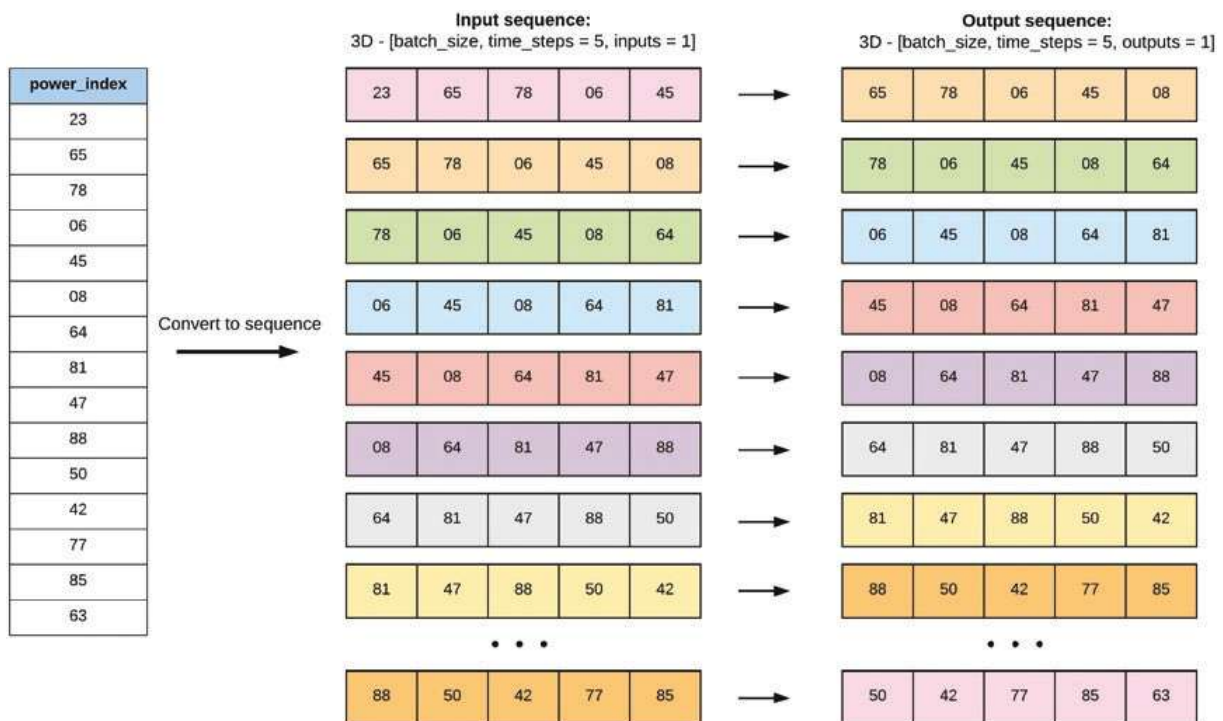


Figure 36-19. Converting a univariate series into sequences for prediction with RNNs. Left: Sample univariate dataset. Center: Input sequence. Right: Output sequence

When modeling using RNNs, it is important to scale the dataset to have values within the same range. The plot in Figure 36-20 shows predictions of the model along with the original targets and the lagging training instances. The next plots in Figure 36-21 and Figure 36-22 show the original series and the RNN generated series in both the scaled and normal values.

For increased training speed, the model will train on a GPU. If running the code on Google Colab, change the runtime type to GPU and install TensorFlow 2.0 with GPU package.

```
# import TensorFlow 2.0 with GPU
!pip install -q tf-nightly-gpu-2.0-preview

# import packages
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# confirm tensorflow can see GPU
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
    raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))

# data file path
file_path = "nigeria-power-consumption.csv"

# load data
parse_date = lambda dates: pd.datetime.strptime(dates, '%d-%m')
data = pd.read_csv(file_path, parse_dates=['Month'], index_col='Month',
                   date_parser=parse_date,
                   engine='python', skipfooter=2)

# print column name
data.columns

# change column names
data.rename(columns={'Nigeria power consumption': 'power-consumption'},
            inplace=True)

# split in training and evaluation set
data_train, data_eval = train_test_split(data, test_size=0.2,
                                         shuffle=False)

# MinMaxScaler - center and scale the dataset
scaler = MinMaxScaler(feature_range=(0, 1))
data_train = scaler.fit_transform(data_train)
data_eval = scaler.fit_transform(data_eval)

# adjust univariate data for timeseries prediction
def convert_to_sequences(data, sequence, is_target=False):
    temp_df = []
    for i in range(len(data) - sequence):
        if is_target:

```

```

        temp_df.append(data[(i+1): (i+1) + sequence])
    else:
        temp_df.append(data[i: i + sequence])
    return np.array(temp_df)

# parameters
time_steps = 20
batch_size = 50

# create training and testing data
train_x = convert_to_sequences(data_train, time_steps, is_target=False)
train_y = convert_to_sequences(data_train, time_steps, is_target=True)

eval_x = convert_to_sequences(data_eval, time_steps, is_target=False)
eval_y = convert_to_sequences(data_eval, time_steps, is_target=True)

# build model
model = tf.keras.Sequential()
model.add(tf.keras.layers.LSTM(128, input_shape=train_x.shape[1:],
                                return_sequences=True))
model.add(tf.keras.layers.Dense(1))

# compile the model
model.compile(loss='mean_squared_error',
              optimizer='adam',
              metrics=['mse'])

# print model summary
model.summary()

# create dataset pipeline
train_ds = tf.data.Dataset.from_tensor_slices(
    (train_x, train_y)).shuffle(len(train_x)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((eval_x, eval_y)).batch(batch_size)

# train the model
history = model.fit(train_ds, epochs=10,
                    steps_per_epoch=500)

```



```

# evaluate the model
loss, mse = model.evaluate(test_ds)

print('Test loss: {:.4f}'.format(loss))
print('Test mse: {:.4f}'.format(mse))

# predict
y_pred = model.predict(eval_x)

# plot predicted sequence
plt.title("Model Testing", fontsize=12)
plt.plot(eval_x[0,:,0], "b--", markersize=10, label="training instance")
plt.plot(eval_y[0,:,0], "g--", markersize=10, label="targets")
plt.plot(y_pred[0,:,0], "r--", markersize=10, label="model prediction")
plt.legend(loc="upper left")
plt.xlabel("Time")
plt.show()

```

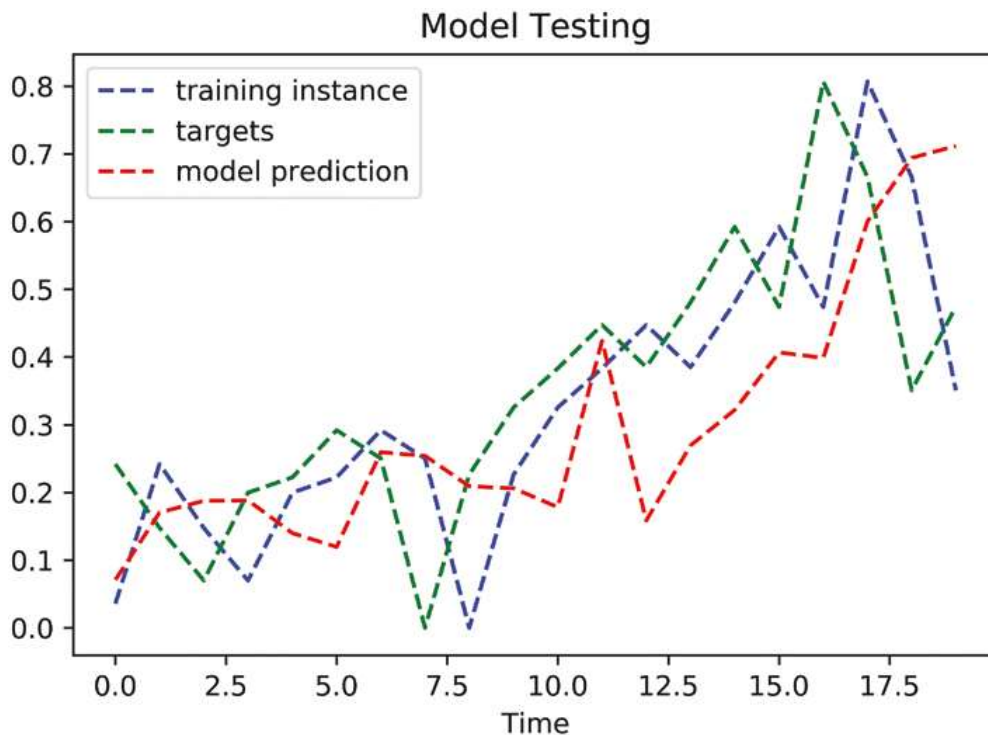


Figure 36-20. Keras LSTM Model Testing


```
# use model to predict sequences using training data as seed
rnn_data = list(data_train[:20])
for i in range(len(data_train) - time_steps):
    batch = np.array(rnn_data[-time_steps:]).reshape(1, time_steps, 1)
    y_pred = model.predict(batch)
    rnn_data.append(y_pred[0, -1, 0])

plt.title("RNN vs. Original series", fontsize=12)
plt.plot(data_train, "b--", markersize=10, label="Original series")
plt.plot(rnn_data, "g--", markersize=10, label="RNN generated series")
plt.legend(loc="upper left")
plt.xlabel("Time")
plt.show()
```

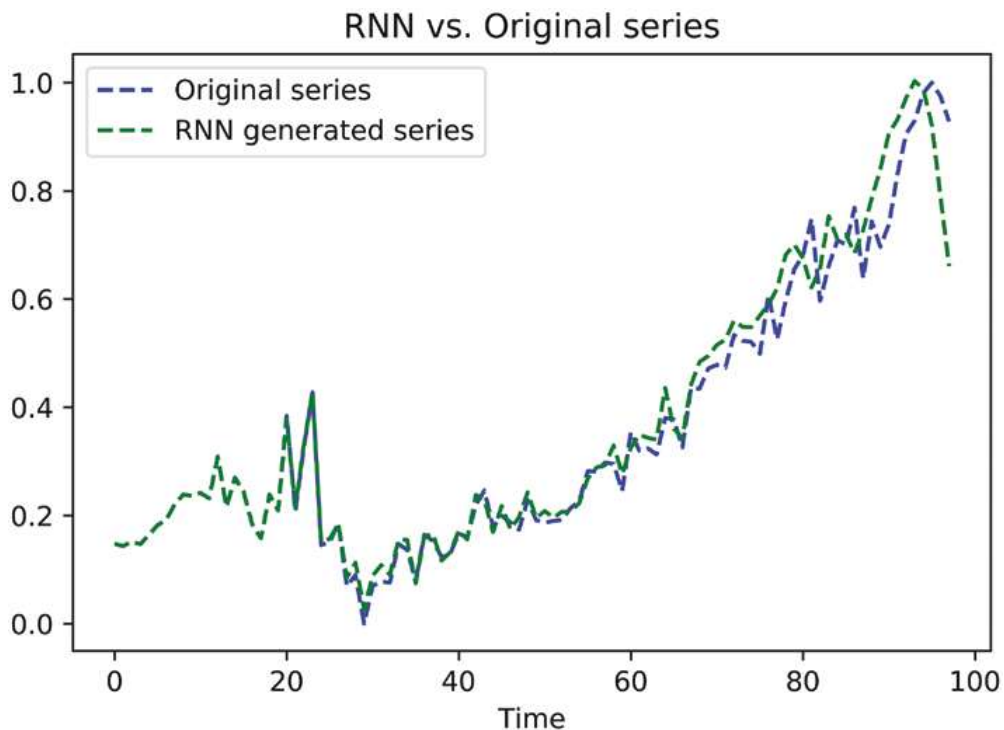


Figure 36-21. Original series vs. RNN generated series – scaled data values

```
# inverse to normal scale and plot
data_train_inverse = scaler.inverse_transform(data_train.reshape(-1, 1))
rnn_data_inverse = scaler.inverse_transform(np.array(rnn_data).reshape(-1, 1))
```

```
plt.title("RNN vs. Original series with normal scale", fontsize=12)
plt.plot(data_train_inverse, "b--", markersize=10, label="Original series")
plt.plot(rnn_data_inverse, "g--", markersize=10, label="RNN generated series")
plt.legend(loc="upper left")
plt.xlabel("Time")
plt.show()
```

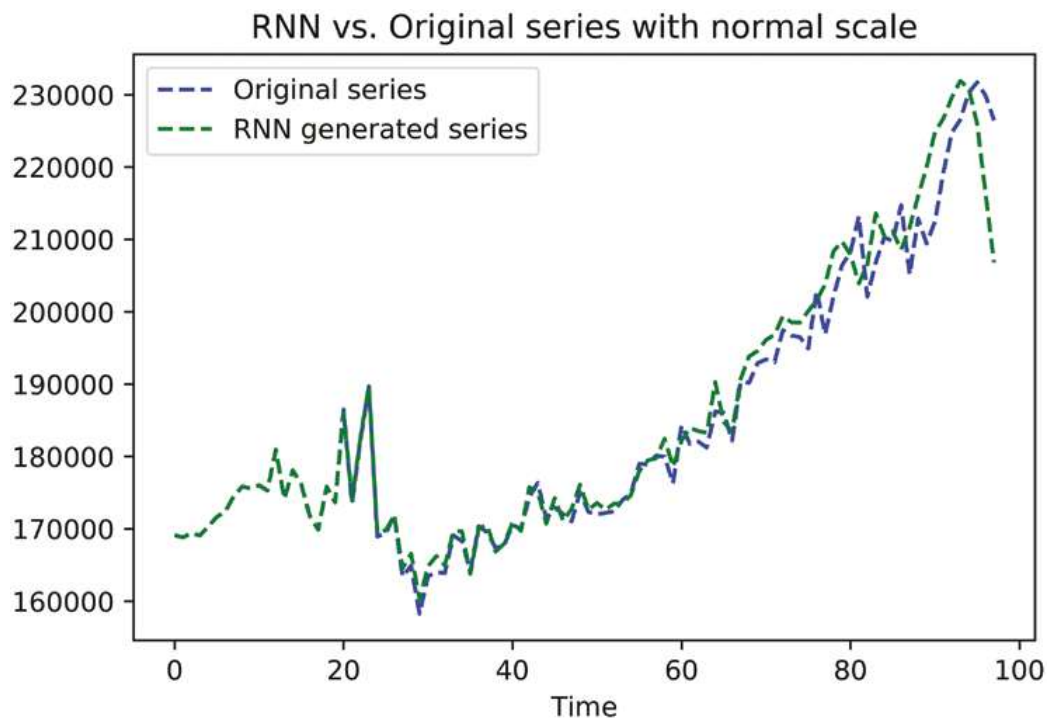


Figure 36-22. Original series vs. RNN generated series – normal data values

From the Keras LSTM code listing, the method `tf.keras.layers.LSTM()` is used to implement the LSTM recurrent layer. The attribute `return_sequences` is set to `True` to return the last output in the output sequence, or the full sequence.

RNN with TensorFlow 2.0: Multivariate Timeseries

The dataset for this example is the Dow Jones Index Data Set from the famous UCI Machine Learning Repository. In this stock dataset, each row contains the stock price record for a week including the percentage of return that stock has in the following week

percent_change_next_weeks_price(). For this example, the record for the previous week is used to predict the percent change in price for the next 2 weeks for Bank of America, BAC stock prices.

The method named **clean_dataset()** carries out some rudimentary cleanup of the dataset to make it suitable for modeling. The actions taken on this particular dataset involve removing the dollar sign from certain of the data columns, removing missing values, and rearranging the data columns so target attribute **percent_change_next_weeks_price** is the last column.

The method named **data_transform()** subselects the stock records belonging to 'Bank of America,' and the target attribute is adjusted so that the previous week record is used to predict the percent change in price for the next 2 weeks. Also, the dataset is split into training and testing sets. The method named **normalize_and_scale()** removes the non-numeric columns and scales the dataset attributes.

Again, the model will train on a GPU instance. The model will be a stacked GRU with multiple GRU layers. This stacking of RNN layers with memory cells makes the network more expressive and can learn more complex long-running sequences. If running the code on Google Colab, change the runtime type to GPU and install TensorFlow 2.0 with GPU package. The output plot in Figure 36-23 is the model predictions showing the targets and the lag training instances.

```
# import TensorFlow 2.0 with GPU
!pip install -q tf-nightly-gpu-2.0-preview

# import packages
import tensorflow as tf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# confirm tensorflow can see GPU
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
    raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))
```

```

# data file path
file_path = "dow_jones_index.data"

# load data
data = pd.read_csv(file_path, parse_dates=['date'], index_col='date')

# print column name
data.columns

# print column datatypes
data.dtypes

# parameters
outputs = 1
stock = 'BAC' # Bank of America

def clean_dataset(data):
    # strip dollar sign from `object` type columns
    col = ['open', 'high', 'low', 'close', 'next_weeks_open', 'next_weeks_
close']
    data[col] = data[col].replace({'\$': ""}, regex=True)
    # drop NaN
    data.dropna(inplace=True)
    # rearrange columns
    columns = ['quarter', 'stock', 'open', 'high', 'low', 'close', 'volume',
'percent_change_price', 'percent_change_volume_over_last_wk',
'previous_weeks_volume', 'next_weeks_open', 'next_weeks_close',
'days_to_next_dividend', 'percent_return_next_dividend',
'percent_change_next_weeks_price']
    data = data[columns]
    return data

def data_transform(data):
    # select stock data belonging to Bank of America
    data = data[data.stock == stock]
    # adjust target(t) to depend on input (t-1)
    data.percent_change_next_weeks_price = data.percent_change_next_weeks_
price.shift(-1)

```

```

# remove nans as a result of the shifted values
data = data.iloc[:-1,:]
# split quarter 1 as training data and quarter 2 as testing data
train_df = data[data.quarter == 1]
test_df = data[data.quarter == 2]
return (np.array(train_df), np.array(test_df))

def normalize_and_scale(train_df, test_df):
    # remove string columns and convert to float
    train_df = train_df[:,2:].astype(float,copy=False)
    test_df = test_df[:,2:].astype(float,copy=False)
    # MinMaxScaler - center and scale the dataset
    scaler = MinMaxScaler(feature_range=(0, 1))
    train_df_scale = scaler.fit_transform(train_df[:,2:])
    test_df_scale = scaler.fit_transform(test_df[:,2:])
    return (scaler, train_df_scale, test_df_scale)

# clean the dataset
data = clean_dataset(data)

# select Dow Jones stock and split into training and test sets
train_df, test_df = data_transform(data)

# scale the data
scaler, train_df_scaled, test_df_scaled = normalize_and_scale(train_df,
test_df)

# split train/ test
train_X, train_y = train_df_scaled[:, :-1], train_df_scaled[:, -1]
test_X, test_y = test_df_scaled[:, :-1], test_df_scaled[:, -1]

# reshape inputs to 3D array
train_X = train_X[:,None,:]
test_X = test_X[:,None,:]

# reshape outputs
train_y = np.reshape(train_y, (-1,outputs))
test_y = np.reshape(test_y, (-1,outputs))

```

```

# model parameters
batch_size = int(train_X.shape[0]/5)
length = train_X.shape[0]

# build model
model = tf.keras.Sequential()
model.add(tf.keras.layers.GRU(128, input_shape=train_X.shape[1:],
                               return_sequences=True))
model.add(tf.keras.layers.GRU(100, return_sequences=True))
model.add(tf.keras.layers.GRU(64))
model.add(tf.keras.layers.Dense(1))

# compile the model
model.compile(loss='mean_squared_error',
              optimizer='adam',
              metrics=['mse'])

# print model summary
model.summary()

# create dataset pipeline
train_ds = tf.data.Dataset.from_tensor_slices(
    (train_X, train_y)).shuffle(len(train_X)).repeat().batch(batch_size)
test_ds = tf.data.Dataset.from_tensor_slices((test_X, test_y)).batch(batch_size)

# train the model
history = model.fit(train_ds, epochs=10,
                    steps_per_epoch=500)

# evaluate the model
loss, mse = model.evaluate(test_ds)

print('Test loss: {:.4f}'.format(loss))
print('Test mse: {:.4f}'.format(mse))

# predict
y_pred = model.predict(test_X)

# plot
plt.figure(1)

```

```

plt.title("Keras - GRU RNN Model Testing for '{}' stock".format(stock),
          fontsize=12)
plt.plot(test_y, "g--", markersize=10, label="targets")
plt.plot(y_pred, "r--", markersize=10, label="model prediction")
plt.legend()
plt.xlabel("Time")
plt.show()
# plt.savefig('gru-bac-model.png', dpi=800)

```

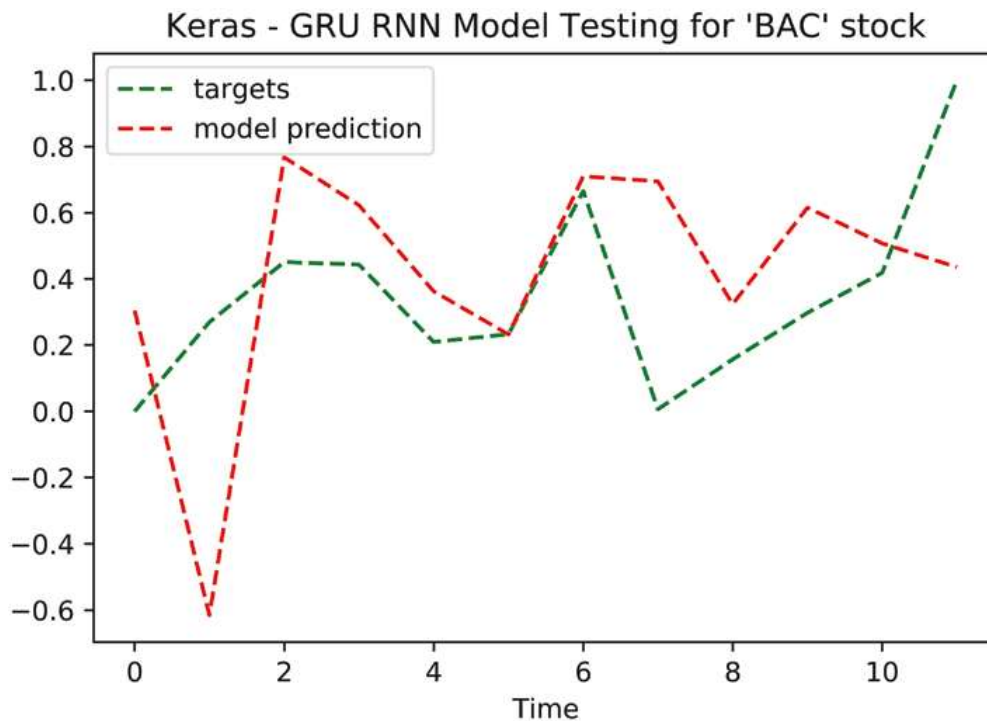


Figure 36-23. GRU RNN Model Testing for Bank of America stock

This chapter gave an overview of recurrent neural networks (RNNs) and its application in learning recurrent models for different types of sequence problems. The next chapter will discuss how we can use neural networks to reconstruct the inputs as some form of unsupervised learning using autoencoders.